

UGANDA CHRISTIAN UNIVERSITY

A Center of Excellence in the Heart of Africa

DEPARTMENT OF COMPUTING & TECHNOLOGY

Faculty of Engineering, Design & Technology

UGANDA CHRISTIAN UNIVERSITY

Faculty of Engineering, Design and Technology

Department of Computing and Technology

WEB SCRAPING, DATA CLEANING AND TEMPORAL ANALYSIS OF ALLBIRDS PRODUCT DATA

Project Report

Course Unit:	DSC2107 – Data Mining and Wrangling
Project:	Web Scraping and Repeated Collection and Analysis of Allbirds Product Data
Group:	Group 9
Programme:	Bachelor of Science in Data Science and Analytics
Reporting Period:	15 October 2026 – 19 October 2026
Group Members:	Agenorwot Cynthia Jillian; Ayoo Alice; Ngobi Jonah; Jjumba; Keji Flora

Mukono, Uganda

Abstract

This project focuses on collecting, cleaning, storing, and analysing publicly accessible product information from the Allbirds e-commerce website. Python was used to retrieve and process product data, Pandas supported cleaning and transformation, SQLite stored product information and time-stamped observations, and Matplotlib produced visualisations for exploratory analysis.

A total of seven raw JSON snapshots were collected across five distinct calendar dates (15–19 October 2026), yielding 692 unique products and 52,178 variant-level observations with no missing prices or identifiers. The average variant price was 101.84 units (median 105, range 0.80–165) and approximately 31.5% of variant observations were available across the period. Of 7,454 variants with first and last valid observations, 70 changed availability status.

The findings are descriptive and limited to the five days of successful collection. The intended seven consecutive days of snapshots were not fully achieved, and this gap is documented as a limitation.

Keywords: Web scraping, data cleaning, Python, Pandas, SQLite, exploratory data analysis, e-commerce.

Chapter 1

Introduction

1.1 Background

E-commerce websites contain product information that can be analysed to understand product characteristics, pricing, and availability. Web scraping retrieves information from websites through automated requests. Combined with data cleaning, database management, and visualisation, it transforms raw web data into an organised dataset suitable for analysis. This project applies these techniques to Allbirds product information.

1.2 Problem Statement

Product information may contain missing values, inconsistent formats, duplicate records, and changing availability. Without a systematic collection and cleaning process, comparisons over time become difficult. This project develops a reproducible pipeline intended to preserve raw data, produce cleaned datasets, maintain historical observations, and generate visual evidence.

1.3 Objectives

1.3.1 Main Objective

To develop a Python-based web-scraping and data-analysis pipeline for collecting, cleaning, storing, and analysing Allbirds product information over a repeated-observation period.

1.3.2 Specific Objectives

1. Examine the target website and determine an appropriate, permitted access method.
2. Develop a scraper to retrieve a small sample and, where permitted, collect available product data.
3. Conduct repeated data collection across the project period.
4. Clean and transform data using Python and Pandas.
5. Design a relational SQLite database for product details and time-stamped observations.
6. Validate the dataset for missing values, duplicates, and inconsistencies.
7. Produce labelled visualisations of product characteristics, prices, and availability.
8. Investigate whether product-variant availability changes during the observation period.

1.4 Scope

The project focuses on product and variant information exposed through the target website's feed, subject to access permissions and actual response structure. Intended fields included identifiers, titles, product types, tags, URLs, variant titles, sizes, prices, availability, and timestamps where available. The project excludes private customer data and bypassing access controls.

Chapter 2

Methodology

2.1 Tools and Technologies

Tool	Purpose
Python	Main programming language
Requests	HTTP requests
urllib.robotparser	Inspect robots.txt rules
Pandas	Data cleaning and transformation
Regular expressions	Price/text cleaning
JSON	Preserve and read raw snapshots
SQLite	Relational data storage
Matplotlib	Data visualisation
VS Code / Jupyter Notebook	Development and testing

2.2 Day 1: Preliminary Scraping Practice

Books to Scrape was used to practise extracting titles, prices, availability, ratings, and product URLs from listing pages and following pagination. The script collected **20 records** before the site stopped responding; the run raised a `ConnectionError` (read timeout) on a subsequent page request.

The practice confirmed that the extraction logic (title, price, availability, star-rating, product URL, and "next" pagination) works correctly, but also highlighted the need for retry handling and controlled delays, which were subsequently incorporated into the Allbirds scraper.

Evidence: 20 records collected; output file `data/raw/books_practice.csv` (not written because the run aborted before the write step); error type: `requests.exceptions.ConnectionError` (HTTPS read timeout on `books.toscrape.com`).

2.3 Day 2: Website Reconnaissance

The reconnaissance notebook requested robots.txt, saved its contents to `data/raw/robots_findings.tx` verified that the products feed was permitted for the project User-Agent, and inspected a small product-feed sample.

Evidence:

- robots.txt saved successfully.
- Feed URL: https://www.allbirds.com/products.json.
- Feed allowed for the project User-Agent: **True**.
- Feed HTTP status: **200**; Content type: `application/json; charset=utf-8`.
- Top-level JSON key: `products`.
- Sample product keys included `id`, `title`, `handle`, `vendor`, `product_type`, `tags`, `variants`, `options`.
- Sample variant keys included `id`, `title`, `option1`–`option3`, `available`, `price`, `compare_at_price`.

2.4 Day 3: Scraper Development

The scraper was designed to request permitted product data, use timeouts and controlled delays (random 3–5 seconds between pages), and preserve raw JSON snapshots with UTC timestamps.

Pagination was verified to work: pages 1, 2 and 3 returned 250, 250 and 192 products respectively, for a total of 692 unique products. The scraper stops if access is denied (HTTP 401/403/429) or if a page returns no new products.

2.5 Days 4–10: Repeated Collection

Collection was attempted across the project period. Successful snapshots were written on **2026-10-15, 2026-10-16 (two runs), 2026-10-17 (two runs), 2026-10-18, and 2026-10-19**, giving **5 distinct calendar dates** of valid data from **7 raw snapshot files**.

The intended seven consecutive days were not fully achieved; missing dates are documented as a limitation and were not fabricated.

2.6 Data Cleaning

The cleaning process handled missing identifiers and timestamps, converted prices to numeric form, extracted variants, identified sizes from product options, standardised tags, and removed duplicate observation keys on `(product_id, variant_id, scraped_at)`.

Cleaned outputs are `products_cleaned.csv` and `observations_cleaned.csv`, and the SQLite database was populated with upsert semantics (products) and insert-or-ignore semantics (observations) so reruns do not duplicate records.

2.7 Database Design

The SQLite database contains two related tables.

2.7.1 Products Table

Field	Description
product_id	Unique product identifier
title	Product title
product_type	Product type
vendor	Vendor, if supplied
tags	Product tags
handle	Product URL handle
product_url	Product page URL

2.7.2 Observations Table

Field	Description
observation_id	Unique observation ID
product_id	Related product ID
scraped_at	Collection timestamp
variant_id	Variant identifier
variant_title	Variant title
size	Size, where identifiable
price	Numeric price
available	Availability indicator, if supplied

The product ID links observations to products. A uniqueness constraint on product ID, variant ID, and timestamp prevents duplicate observations for the same variant at the same time.

2.8 Data Validation

Validation checks recorded counts, missing values, duplicate keys, invalid prices, availability values, timestamps, and relationships between tables.

2.9 Exploratory and Temporal Analysis

The generated charts are: price distribution; average price by product type; availability counts; average recorded price by date; available variant observations by date; price distribution by availability; plus an availability-percentage-by-day line chart and a first-versus-last availability comparison per variant.

Chapter 3

Data Management and Analysis

3.1 Data Storage

Raw JSON snapshots are kept separately from cleaned CSV files, while SQLite stores product details and historical observations. This supports reproducibility and allows cleaning decisions to be revisited.

3.2 Data Quality Assessment

Measure	**Actual Result**
Raw snapshots processed	7
Unique products	692
Variant observations	52,178
Missing prices	0
Duplicate observation keys	0 (after deduplication on product/variant/timestamp)
Distinct collection dates	5
Invalid/missing identifiers	0

Additional quality observations:

- No missing values were found in either the `products` or `observations` tables.
- Price range 0.80–165 with mean 101.84, median 105, IQR 75–130.

- Product type counts: Shoes 602, Socks 57, Apparel 22, blank 7, Underwear 3, `return,package_protection` 1. The blank category and the non-product `return,package_prote` entry are data-quality artefacts that should be excluded from type-level averages.
- Availability split: 35,772 unavailable vs 16,406 available (about 31.5% available).

3.3 Visualisation and Interpretation

The price distribution chart (Figure 1, `01_price_distribution.png`) shows prices concentrated between 75 and 130, with a long left tail to a minimum of 0.80 and a right tail to 165.

The average-price-by-type chart (Figure 2) shows differences between categories, but the blank and `return,package_protection` categories should be treated with caution.

The availability chart (Figure 3) shows that most variant observations were unavailable (35,772 vs 16,406).

The daily average price chart (Figure 4) and the daily available-variant observations chart (Figure 5) provide a five-date temporal view; availability remained relatively stable between roughly 31.1% and 31.7% (Figure 7, `availability_percentage_by_day.png`).

The price-by-availability box plot (Figure 6) compares price distributions for available and unavailable variants.

An additional first-versus-last comparison showed that **70 of 7,454** variants (about 0.94%) changed availability status between their earliest and latest valid observations, while 7,384 did not. This is a small but non-zero short-term change signal. Missing observations are not automatically treated as unavailable.

Actual findings:

1. Five distinct collection dates.
2. 692 products and 52,178 variant observations.
3. Mean price 101.84.
4. Availability around 31.5% overall and stable across days.
5. 70 variants with changed status between first and last observation.

These are descriptive findings only and do not establish long-term trends.

Chapter 4

Ethical Considerations and Limitations

4.1 Ethical Considerations

The project is designed around publicly accessible product information and responsible collection. The group checked robots.txt and applicable restrictions, controlled request frequency (3–5 seconds random delays between pages), and stopped when access was refused. It did not attempt to bypass controls or retrieve private customer information.

4.2 Limitations

The feed may restrict access, return errors, or change format. Pagination parameters may not work as expected. Only five distinct calendar dates yielded valid snapshots, not the intended seven.

The practice scrape of Books to Scrape timed out mid-run, which prevented its CSV from being written. The price minimum of 0.80 and the `return,package_protection` product type suggest some non-standard records.

Seven days (or fewer, as here) provides a short-term view rather than evidence of long-term trends. Results describe only the records actually returned and should not automatically be treated as a complete catalogue.

Chapter 5

Conclusion and Recommendations

5.1 Conclusion

The project applies web scraping, data cleaning, relational database design, and exploratory analysis to Allbirds product information.

Seven JSON snapshots across five distinct dates produced 692 products and 52,178 variant observations with no missing prices or identifiers. Availability was around 31.5% overall and 70 variants changed status between first and last observation.

JSON snapshots support preservation, Pandas supports transformation, and SQLite relates product details to time-stamped observations.

5.2 Recommendations

Verify website access rules and endpoint behaviour before repeated collection. Preserve raw responses, record timestamps consistently, document failed attempts, validate data before analysis, and ensure every chart has a title, labelled axes, and evidence-based interpretation.

Add retry logic and longer timeouts to the practice scraper; remove the stray phone-number/placeholder logic from `scraper.ipynb`; and treat blank or non-product `product_type` values as separate from real category averages.